

Homework 7: When Your Startup's Flash Sale Almost Failed

Author: Nithish Bhat

Course: CS6650 — Distributed Systems

Date: March 14, 2026

Part I: Building on Logical Clocks

Vector Clocks vs Lamport's Logical Clocks

Problem Lamport Clocks Cannot Solve:

Consider three processes P1, P2, P3. P1 sends message A to P2, and independently P3 sends message B to P2. With Lamport clocks, P2 might assign timestamps such that A has timestamp 3 and B has timestamp 4. Looking at these timestamps alone, it appears A happened before B. However, A and B are actually **concurrent events** — P1 and P3 acted independently with no causal relationship.

Lamport clocks cannot distinguish "happened before" from "concurrent." They only guarantee: if $A \rightarrow B$, then $L(A) < L(B)$. But $L(A) < L(B)$ does **not** imply $A \rightarrow B$.

How Vector Clocks Solve This:

Vector clocks maintain a vector of counters, one per process. When P2 receives messages A and B, their vector timestamps would be:

- A arrives with vector $[2, 0, 0]$ (only P1's counter advanced)
- B arrives with vector $[0, 0, 2]$ (only P3's counter advanced)

Since neither vector dominates the other ($2 > 0$ but $0 < 2$), the vector clock correctly identifies A and B as **concurrent**. This is critical for conflict detection in distributed databases and collaborative systems.

Part II: Synchronous vs Asynchronous Order Processing

Phase 1: Synchronous Bottleneck

Implementation:

The API exposes a `POST /orders/sync` endpoint. A buffered channel with capacity 15 acts as a semaphore to simulate the payment processor bottleneck. Each payment verification acquires a

semaphore slot, sleeps for 3 seconds, and releases the slot. With capacity 15, the system supports 5 concurrent completions per second ($15 \text{ slots} \div 3 \text{ seconds} = 5/\text{sec}$).

Normal Load Test (5 users, 30 seconds):

Metric	Value
Total Requests	40
Failures	0 (0%)
Avg Response Time	3,046 ms
P95 Latency	3,100 ms
P99 Latency	3,100 ms
RPS	1.35

Under normal load, all requests complete successfully with consistent ~3s response times. The semaphore never saturates.

Flash Sale Test (20 users, 60 seconds):

Metric	Value
Total Requests	285
Failures	0 (0%)
Avg Response Time	3,678 ms
P95 Latency	4,900 ms
P99 Latency	5,100 ms
RPS	4.90

With 20 concurrent users, the system degraded significantly. While no outright failures occurred (the ALB's 60-second timeout was never exceeded), latency increased by 70% at the tail. Throughput was hard-capped at ~5 RPS regardless of demand. At higher concurrency (60+ users as described in the scenario), requests would begin timing out and returning 504 errors.

Phase 2: Bottleneck Analysis

- **Payment processor speed:** 1 order per 3 seconds per slot
- **Semaphore capacity:** 15 slots
- **Maximum throughput:** $15 \div 3 = 5$ orders/second
- **Flash sale demand:** ~55 orders/second (observed from async test)
- **Orders lost per second:** $55 - 5 = \sim 50$ orders/second

The payment processor is the immovable bottleneck. We cannot make it faster. What we **can** change is how customers experience the wait — instead of blocking their HTTP connection, we acknowledge immediately and process in the background.

Phase 3: Asynchronous Solution

Architecture:

```
Sync:   Customer → API → Payment (3s) → Response
Async:  Customer → API → SNS → Response (<100ms)
                        ↓
                        SQS → Workers → Payment (3s)
```

The `POST /orders/async` endpoint publishes the order to an SNS topic and immediately returns `202 Accepted`. SNS fans out to an SQS queue, where background worker goroutines poll and process orders.

Flash Sale Test (20 users, 60 seconds):

Metric	Value
Total Requests	3,289
Failures	0 (0%)
Avg Response Time	57 ms
P95 Latency	88 ms
P99 Latency	190 ms
RPS	54.97

Comparison:

Metric	Sync Flash	Async Flash	Improvement
RPS	4.90	54.97	11.2x
Avg Latency	3,678 ms	57 ms	64.5x faster
P95 Latency	4,900 ms	88 ms	55.7x faster
Total Orders	285	3,289	11.5x more

Speedup = T_sync / T_async = 3,678 / 57 = **64.5x**

Phase 4: The Queue Problem

With a single worker goroutine processing 1 order every 3 seconds:

- **Acceptance rate:** ~55 orders/second
- **Processing rate:** 0.33 orders/second
- **Queue growth rate:** 55 - 0.33 = **~54.67 messages/second**
- **Peak queue depth observed:** ~3,240 messages

After a 60-second flash sale, approximately 3,240 messages accumulated. At 0.33 messages/second drain rate, clearing the backlog would take: 3,240 ÷ 0.33 = **~9,818 seconds (~2.7 hours)**.

CloudWatch screenshot shows the queue spike to ~3.24K during the flash sale with 1 worker.

Phase 5: Scaling Workers

All tests conducted with 20 Locust users at 10 users/second spawn rate for 60 seconds. Queue was purged between tests.

Worker Goroutines	Processing Rate	Peak Queue Depth	Drain Behavior
1	0.33/sec	~3,240	Hours to drain
5	1.67/sec	~2,880	Very slow drain
20	6.67/sec	~2,880	Moderate drain
100	33.3/sec	~33	Near-instant drain

Observations:

With 1-20 workers, the queue still grew rapidly because processing rate remained far below the ~55/sec incoming rate. At 100 workers (33.3/sec processing), the queue barely accumulated — peaking at only 33 messages. The "Number Of Empty Receives" metric spiked to ~302, indicating workers were frequently outpacing the incoming messages.

Minimum workers to prevent queue buildup:

At 55 orders/second incoming, each worker processes 1 order per 3 seconds. Minimum workers needed: $55 \times 3 = 165$ goroutines. In practice, ~180 goroutines would provide a comfortable buffer to match the incoming rate and prevent sustained queue growth.

Analysis Questions

How many times more orders did async accept vs sync?

Async accepted 3,289 orders vs sync's 285 — approximately **11.5x more** in the same time period.

What causes queue buildup and how do you prevent it?

Queue buildup occurs when the ingestion rate exceeds the processing rate. Prevention strategies include: scaling worker goroutines to match incoming throughput, scaling ECS tasks horizontally (multiple task instances), or implementing auto-scaling policies based on the `ApproximateNumberOfMessagesVisible` CloudWatch metric.

When would you choose sync vs async in production?

Sync is appropriate when the client needs an immediate, definitive result (e.g., real-time account balance checks, authentication). Async is preferred when the operation is time-consuming, the client doesn't need the result immediately, and you need to absorb traffic spikes gracefully (e.g., order processing, email sending, report generation).

Part III: Serverless with AWS Lambda

Architecture Change

Part II: Order API → SNS → SQS → ECS Workers (self-managed)
Part III: Order API → SNS → Lambda (AWS-managed)

The Lambda function subscribes directly to the SNS topic, eliminating the SQS queue entirely. AWS manages invocation, scaling, and retry logic.

Cold Start Observations

Lambda was tested by sending 10 orders manually via `curl`. CloudWatch logs revealed:

Invocation Type	Duration	Init Duration	Memory Used
Cold Start #1	3,004.21 ms	72.38 ms	20 MB
Cold Start #2	3,005.42 ms	74.69 ms	20 MB
Cold Start #3	3,005.34 ms	75.08 ms	20 MB
Warm Start #1	3,003.65 ms	—	20 MB
Warm Start #2	3,001.44 ms	—	20 MB
Warm Start #3	3,003.54 ms	—	20 MB

Key findings:

- **Cold start overhead:** ~73ms average (range 72-75ms)
- **Cold start as % of total:** 73ms / 3000ms = ~**2.4%** — negligible for 3-second payment processing
- **Warm start consistency:** ~3001-3004ms, virtually no overhead
- **Automatic scaling:** 4 separate log streams were created for concurrent orders, meaning Lambda spun up 4 independent instances automatically with zero configuration
- **Memory usage:** Only 20MB out of 512MB allocated

Cost Analysis

Part II (ECS) Monthly Cost:

- 2 ECS Fargate tasks (API + Worker) running 24/7
- $2 \times \$8.50/\text{month} = \text{\$17/month}$ (always running, even with zero orders)

Part III (Lambda) Monthly Cost:

Assuming 10,000 orders/month at 3 seconds each with 512MB (0.5GB):

- Requests: 10,000 (within 1M free tier) = **\$0**
- GB-seconds: $10,000 \times 3 \times 0.5 = 15,000$ (within 400K free tier) = **\$0**
- **Monthly cost: \$0 (FREE)**

Break-even calculation:

- Free tier covers 400,000 GB-seconds

- Per order: $3s \times 0.5GB = 1.5 \text{ GB-seconds}$
- Free tier covers: $400,000 \div 1.5 = \sim 267,000 \text{ orders/month}$
- Beyond free tier, reaching \$17/month equivalent requires $\sim 1.7M$ requests/month

Lambda is effectively free for any startup processing under 267K orders/month.

Reliability Trade-offs

Aspect	Part II (ECS + SQS)	Part III (Lambda)
Message Retention	SQS retains for 4 days	SNS retries 2x, then discards
Scaling	Manual goroutine/task tuning	Automatic, up to account limit
Failure Recovery	Messages wait in queue	Lost after 2 retries
Ops Overhead	Monitor queue depth, health, scaling	Zero — no queue, no workers
Cold Starts	N/A (always running)	$\sim 73ms$ (2.4% overhead)

Critical scenario: If the payment API goes down for 2 hours, Part II's SQS queue retains all messages for up to 4 days — once the API recovers, every order is processed. Part III's SNS → Lambda would retry each message twice and then discard it. All orders during the outage would be permanently lost.

Recommendation: Should Your Startup Switch to Lambda?

For an early-stage startup processing fewer than 267K orders/month, Lambda is the clear choice. The cost savings (\$0 vs \$17/month), elimination of operational overhead (no queue monitoring, no worker scaling, no 3am alerts), and automatic scaling far outweigh the trade-offs. The cold start overhead of $\sim 73ms$ is negligible on a 3-second payment operation. The main risk is the loss of SQS's message durability — if the payment processor goes down, orders are lost after 2 retries. This can be mitigated by adding a dead-letter queue or implementing idempotent order tracking in a database. As the startup scales beyond 267K orders/month or requires guaranteed message delivery for compliance reasons, migrating back to the SQS + ECS architecture (or a hybrid approach) would be warranted.

Infrastructure Summary

Terraform-managed resources:

- VPC (10.0.0.0/16) with 2 public subnets (ALB) and 2 private subnets (ECS)
- NAT Gateway for private subnet internet access
- Application Load Balancer with health checks on `/health`
- ECR repositories for API and Worker images
- ECS Cluster with 2 Fargate services (API + Worker)
- SNS Topic: `order-processing-events`
- SQS Queue: `order-processing-queue` (visibility timeout 30s, long polling 20s)
- Lambda function: `hw7-order-processor` (512MB, provided.al2 runtime)

Application code:

- `api/main.go` — Go HTTP server with `/health`, `/orders/sync`, `/orders/async`
- `worker/main.go` — SQS poller with configurable goroutine count via `NUM_WORKERS`
- `lambda/main.go` — SNS-triggered Lambda handler

Load testing:

- Locust with `SyncUser` and `AsyncUser` classes
- Configurable user count, spawn rate, and duration